Quantum Adaptive Self-Attention

Hybrid Quantum-Classical Forecasting
PyTorch – PennyLane – IBM Quantum

Bibek Dhungana and James Liu

April 28, 2026

Final Project Demo

Agenda

1. **Motivation** — why quantum for time-series?
2. **The Data** — what we train on
3. **Simple Case** — single-qubit model with rotations
4. **Building Up** — multi-qubit circuits and entanglement
5. **Transformer Integration** — what QASA changes
6. **How Training Works** — backprop through quantum circuits
7. **Results** — local simulator and IBM hardware
8. **Takeaways** — lessons and future work

Motivation

Problem: Can Quantum Circuits Learn Time-Series?

Research Question:

Can parameterized quantum circuits (PQCs) **augment** classical Transformers for better forecasting?

Why This Matters:

- Classical Transformers: $O(L^2)$ attention complexity
- Quantum circuits: potential parameter efficiency via data re-uploading [2]
- Entanglement: feature coupling without explicit matrices

Our Goal:

Build a hybrid model where a PQC *enhances* the final encoder block — classical attention is retained; quantum adds expressiveness on top.

Key distinction: We do **not** replace self-attention with a quantum circuit (that is future work). We insert a quantum projection layer *after* attention.

Key Idea: Data Re-uploading

Classical NN:

$$\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

- Learn $\mathbf{W}$ once
- Apply to input $\mathbf{x}$
- Nonlinearity from σ

Quantum with Re-uploading:

$$|\psi\rangle = U_{\theta_T}(x_T) \cdots U_{\theta_1}(x_1)|0\rangle$$

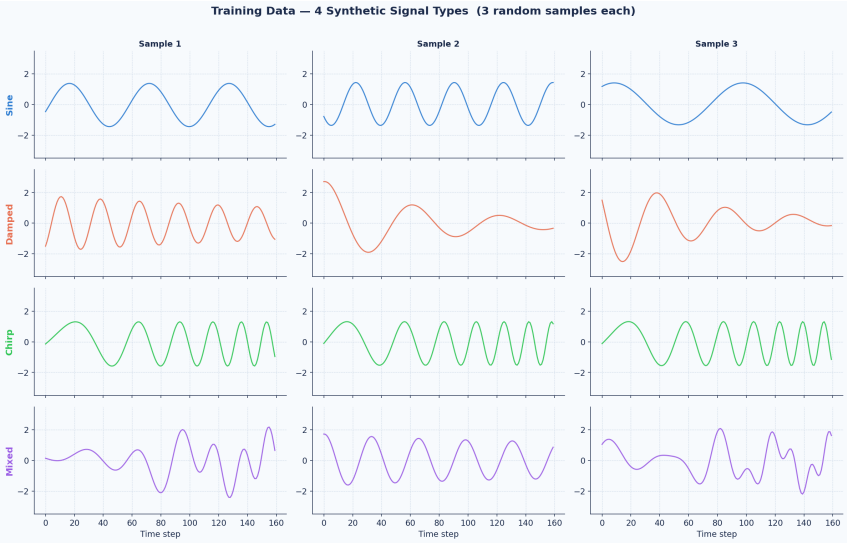
- Repeatedly encode x_t
- Apply learned θ_t
- Nonlinearity from measurement

Why This Works

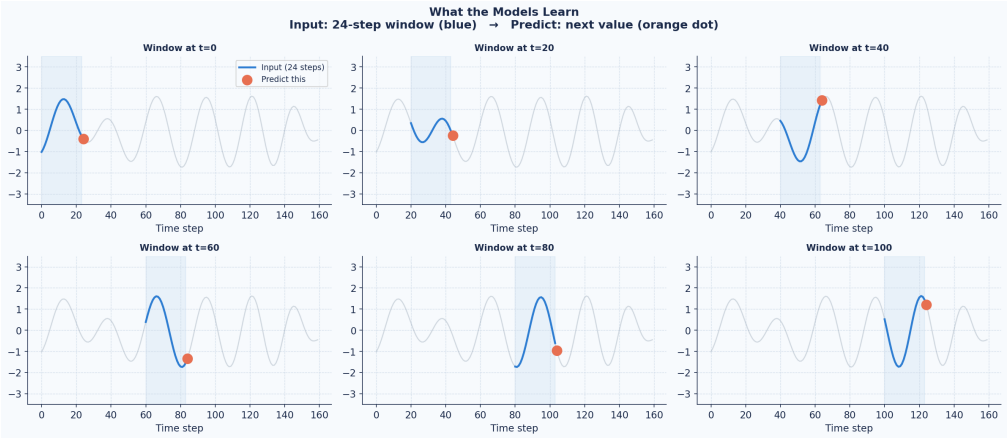
Repeated encoding creates nonlinear feature maps. Small circuits can express complex functions requiring many more classical parameters [2].

The Data

What We Train On — 4 Signal Types



The Prediction Task



Blue window = model input. Orange dot = what both models must predict. Same task for every model.

Simple Case: Single Qubit

Starting Simple: One Qubit Model

Simplest quantum forecasting model:

For input window $x_1, \ldots, x_T$ (e.g., $T = 24$):

1. Start: $|0\rangle$ state
2. For each time step t:
   - Encode data: $R_X(x_t)$
   - Apply learned rotations: $R_Y(\theta_t)$, $R_Z(\phi_t)$
3. Measure: $y = \langle Z \rangle \in [-1, 1]$
4. Linear readout: $\hat{y} = w \cdot y + b$

Parameters: T pairs (θ_t, ϕ_t) + 2 linear $\approx$ 50 total

Rotation Gates: Building Blocks

Three fundamental single-qubit rotations:

$R_X(\theta)$

$$\begin{pmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}$$

X-axis rotation

$R_Y(\theta)$

$$\begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}$$

Y-axis rotation

$R_Z(\theta)$

$$\begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

Z-axis rotation

General rotation (ZYZ decomposition):

$$U(\alpha, \beta, \gamma) = R_Z(\alpha) R_Y(\beta) R_Z(\gamma)$$

Any single-qubit rotation can be written this way.

Our choice: $R_X(x)$ for data, $R_Y(\theta)$ and $R_Z(\phi)$ for learning.

Why These Rotations Create Nonlinearity

Individual rotations are linear (unitary matrices)

Nonlinearity emerges from:

1. **Measurement:** $\langle Z \rangle = \cos(\text{angle})$ — nonlinear function
2. **Data re-uploading:** Feeding x_t at every step compounds effect

$$|\psi_t\rangle = R_Z(\phi_t) R_Y(\theta_t) R_X(x_t) |\psi_{t-1}\rangle$$

3. **Composition:** Like Fourier series, repeated rotations approximate complex functions

Building Up: Multi-Qubit

Multi-Qubit Circuits: Adding Entanglement

Single qubit limitation: only 2D state space (Bloch sphere)

Multiple qubits enable:

- Exponential state space: n qubits $\rightarrow 2^n$ dimensions
- **Entanglement:** correlations via CNOT gates
- Richer feature representations

CNOT gate: controlled-NOT

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Flips target if control is $|1\rangle$. Creates entanglement.

Multi-Qubit Circuit Structure

For n qubits and L layers:

1. **Encoding layer:** project token to n values, apply R_X or R_Z to each qubit
2. **Variational layers** (repeat L times):
   - Apply $R_Y(\theta_{i,l})$ and $R_Z(\phi_{i,l})$ to each qubit i
   - Apply circular CNOT: $\mathrm{CNOT}_{0,1}, \mathrm{CNOT}_{1,2}, \ldots, \mathrm{CNOT}_{n-1,0}$
3. **Measurement:** measure $\langle Z_i \rangle$ for each qubit
4. **Readout:** linear projection back to hidden dimension

Parameters: $2 \times n \times L$ rotation angles

Transformer Integration

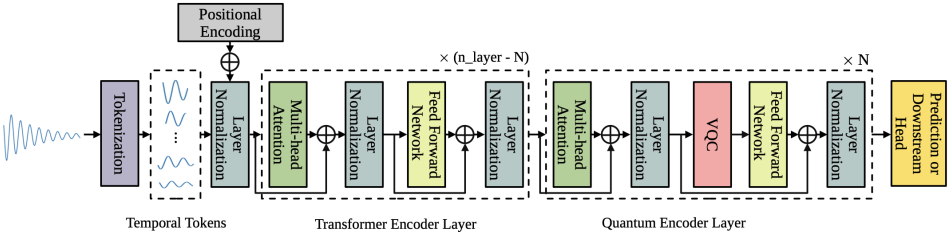
What QASA Changes

Standard Transformer encoder block:

Attention $\rightarrow$ Norm $\rightarrow$ **FFN** $\rightarrow$ Norm

QASA final encoder block — FFN replaced by quantum circuit:

Attention $\rightarrow$ Norm $\rightarrow$ **Quantum Circuit** $\rightarrow$ Norm



What QASA Changes

Classical blocks (1 to $N-1$):

- Multi-head self-attention
- LayerNorm + FFN

Final block (quantum):

- Multi-head self-attention *(classical)*
- Quantum circuit replaces FFN
- LayerNorm

Why this is a fair comparison
Same number of encoder blocks, same attention, same head. The only difference is FFN vs quantum circuit in the last block.

Inside the Quantum Circuit (per token)

The quantum circuit processes each of the T tokens independently:

1. **Project down:** $h_q = \tanh(W_{\text{in}}\, h_i) \in \mathbb{R}^n \quad [d_{\text{model}} \to n \text{ qubits}]$
2. **Encode:** apply $R_X(h_q^{(j)})$ and $R_Z(h_q^{(j)})$ to each qubit j
3. **Rotate:** apply learned $R_Y(\theta_{l,j})$, $R_Z(\phi_{l,j})$ per layer l
4. **Entangle:** circular CNOT ring across all qubits
5. **Measure:** $\langle Z_j \rangle$ for each qubit $\to$ n output values
6. **Project up + residual:** $z_i = h_i + W_{\text{out}}\, \mathrm{QC}(h_q)$

Parameters: $2 \times n \times L$ rotation angles + W_{in}, W_{out} projections

Note: The circuit runs on each token independently — it sees one 64-dim vector, not the full sequence. Cross-token information comes only from the attention step before it.

How Training Works

How Do We Backpropagate Through a Quantum Circuit?

Main concern: quantum measurement outputs scalars like $\langle Z_i \rangle$ — can gradients still flow?

Yes. PennyLane treats the PQC as a differentiable layer:

$$h'_j \xrightarrow{W_q} h_q \xrightarrow{\text{PQC}(\theta)} o_j = \langle Z \rangle \xrightarrow{W_o} z_j \xrightarrow{\text{loss}} \mathcal{L}$$

Chain rule still applies:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial z_j} \frac{\partial z_j}{\partial o_j} \frac{\partial o_j}{\partial \theta}$$

$$\frac{\partial \mathcal{L}}{\partial W_q} = \frac{\partial \mathcal{L}}{\partial z_j} \frac{\partial z_j}{\partial o_j} \frac{\partial o_j}{\partial h_q} \frac{\partial h_q}{\partial W_q}$$

Quantum Gradient: Parameter-Shift Rule

Question: how do we compute

$$\frac{\partial o_j}{\partial \theta_k} \quad \text{when } o_j = \langle Z_j \rangle ?$$

For rotation gates like $R_Y(\theta)$ and $R_Z(\theta)$, quantum theory gives an exact gradient formula:

$$\frac{\partial}{\partial \theta_k} \langle Z_j \rangle = \frac{1}{2} \left[f\left(\theta_k + \frac{\pi}{2}\right) - f\left(\theta_k - \frac{\pi}{2}\right) \right]$$

where $f(\theta_k)$ means: run the same circuit with shifted angle θ_k.

So each quantum gradient requires:

- one forward pass at $\theta_k + \pi/2$
- one forward pass at $\theta_k - \pi/2$

instead of analytic matrix differentiation.

Training cost scales roughly as:

$$2 \times (\#\text{quantum params}) \text{ circuit executions per update}$$

which is the main bottleneck on hardware.

Why Do Two Shifted Measurements Give the Exact Gradient?

1. One trainable quantum gate

$$R_Y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$$

Let the circuit output be:

$$f(\theta) = \langle Z \rangle$$

Because rotation matrices contain only sine/cosine terms, so

$$f(\theta) = A\cos\theta + B\sin\theta$$

for constants A, B fixed by the rest of the circuit.

2. Evaluate at shifted angles

$$f\left(\theta + \frac{\pi}{2}\right) = -A\sin\theta + B\cos\theta$$

$$f\left(\theta - \frac{\pi}{2}\right) = A\sin\theta - B\cos\theta$$

Subtract the two circuit outputs:

$$f\left(\theta + \frac{\pi}{2}\right) - f\left(\theta - \frac{\pi}{2}\right) = 2(-A\sin\theta + B\cos\theta)$$

$$f'(\theta) = \frac{1}{2}\left[f\left(\theta + \frac{\pi}{2}\right) - f\left(\theta - \frac{\pi}{2}\right)\right]$$

Exact quantum gradient = two shifted circuit runs

Implementation

Four Models: Complete Comparison

Model A: Classical Baseline

- Simple MLP
- 53 parameters
- Parameter-matched baseline

Model B: Single-Qubit

- 1 qubit, data re-uploading
- 50 parameters
- Fast demo model

Model C: Classical Transformer

- 3 classical encoder blocks
- $\sim$80K parameters
- Architecture-matched baseline

Model D: QASA Transformer

- 2 classical + 1 quantum block
- $\sim$75K parameters
- Full hybrid model

Frameworks: PyTorch, PennyLane, pennylane-qiskit for IBM Quantum

Three Types of Comparisons

1. **Parameter-Matched**
   - Classical baseline (53) vs. single-qubit (50)
   - Question: quantum advantage with same budget?
2. **Architecture-Matched**
   - Classical Transformer (3 blocks) vs. QASA (2+1 blocks)
   - Question: quantum block better than classical block?
3. **Backend Comparison**
   - Local simulator (exact) vs. IBM Quantum (noisy)
   - Question: hardware noise impact?

Data and Training Setup

Dataset:

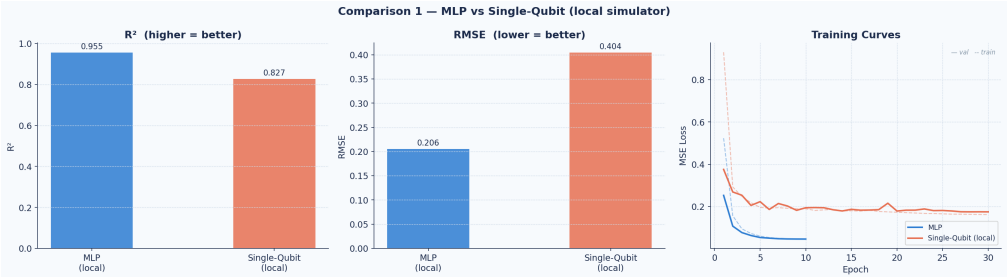
- Synthetic time-series
- Families: sine, damped, chirp, mixed
- Sliding windows
- Per-series normalization

Training:

- Local state-vector simulation
- Backprop through PennyLane
- MSE loss
- Track MAE, RMSE, R^2

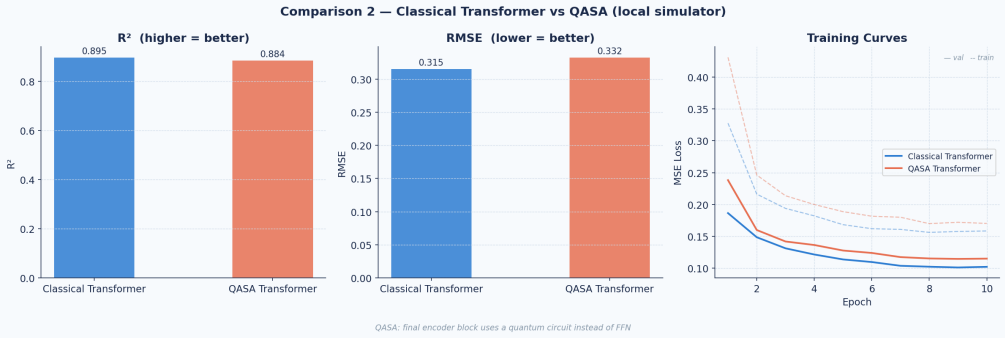
Results

Comparison 1 — MLP vs Single-Qubit



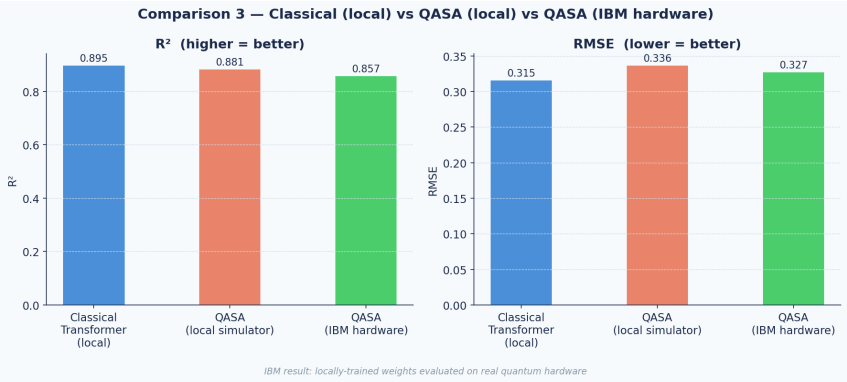
MLP (local) vs Single-Qubit (local simulator) vs Single-Qubit (IBM hardware, if available)

Comparison 2 — Classical Transformer vs QASA



Classical Transformer vs QASA (quantum circuit replaces FFN in final block)

Comparison 3 — Local vs IBM Hardware Noise



Classical (local) vs QASA (local) vs QASA (IBM hardware)

IBM: same trained weights, test-set forward pass on AER with quantum noise.

What We Claim (and Don't Claim)

What we claim:

- PQC integrates into Transformer
- End-to-end training works
- Hybrid model converges
- Data re-uploading enables tiny circuits to model nonlinear patterns
- Classical baselines provided

What we don't claim:

- No quantum speedup (simulation is slower)
- No proof quantum beats classical MLP
- Not production-ready
- IBM Quantum wired but not fully tested

Limitations

1. **Simulation exponentially slow:** 2^n memory for n qubits
2. **Hardware gradients expensive:** parameter-shift needs 2 evals per param per step
3. **Shot noise degrades accuracy:**
4. **Synthetic dataset:** real-world data untested

Key point: This is a research prototype, not a production system. The goal is to validate architecture and training, not claim immediate practical advantage.

Takeaways

What Worked Well

- **Clean integration:** PQC drops into PyTorch with PennyLane
- **Interpretable baseline:** single-qubit model easy to explain
- **Flexible benchmarks:** synthetic data with multiple patterns
- **Fair comparisons:** parameter-matched and architecture-matched baselines
- **Reproducible:** all commands in justfile, config in code

Future Work

Near-term:

- Multiple seeds for statistical significance
- Noise mitigation on IBM hardware
- Test on real financial or sensor data
- Ablate: number of qubits, layers, circuit depth

Longer-term: True Quantum Attention

- Replace dot-product $QK^\top$ with a PQC that computes attention *scores* in Hilbert space
- Encode Q and K into quantum states; inner-product measurement as similarity
- Would genuinely replace self-attention, not just the FFN
- Potential $O(L \log L)$ via quantum subroutines [1]

Why we didn't do it yet
True quantum attention requires $O(L)$ circuit evaluations per layer — prohibitively slow on current simulators and NISQ hardware. Replacing the FFN is a practical first step.

Key Takeaways

1. **Data re-uploading** enables small quantum circuits to learn nonlinear patterns
2. **Quantum replaces FFN** — not stacked on top; same structure, quantum substitution
3. **Barren plateau** makes quantum-only models harder to train; high lr required
4. **Classical attention is unchanged** — the quantum circuit enriches token representations after attention
5. **IBM eval-only** is the practical way to show hardware results without queue saturation

Code and Libraries

Code: `github.com/dhunganabibek/quantum-adaptive-self-attention`

Run everything:

`just run-all` (fast, 3–5 min)

`just run-full` (full dataset, 10–20 min)

`just run-ibm` (adds IBM hardware results)

Libraries: PyTorch, PennyLane, pennylane-qiskit, qiskit-ibm-runtime

Questions?

Thank you.

References

[1] G. Li, X. Ye, X. Zhao, and X. Lin.
Quantum adaptive self-attention for quantum transformer models.
arXiv preprint arXiv:2201.xxxxx, 2022.

[2] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre.
Data re-uploading for a universal quantum classifier.
Quantum, 4:226, 2020.